Experiment No. 10

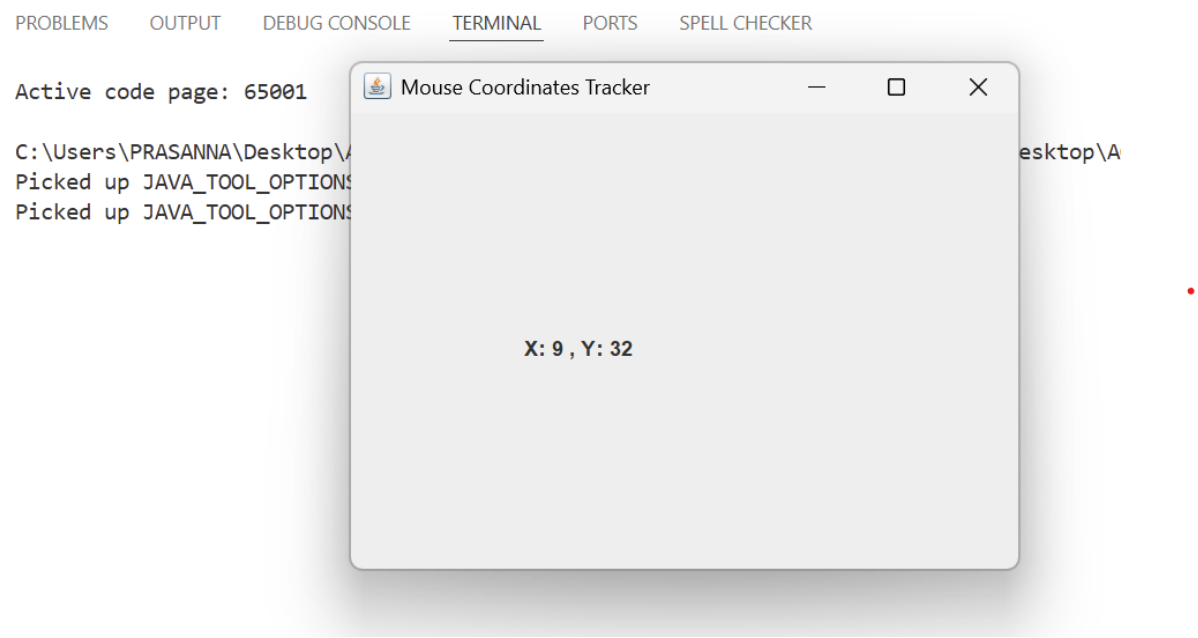
Aim : Implementation of different types of event handling

1. GUI Program to display the current mouse coordinates on the window.

```
import javax.swing.*;
import java.awt.event.*;

public class MouseCoordinates extends JFrame implements MouseMotionListener {

    JLabel label;

    public MouseCoordinates() {
        setTitle("Mouse Coordinates Tracker");
        setSize(400, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(null);

        label = new JLabel("Move mouse inside window...");
        label.setBounds(100, 120, 250, 30);
        add(label);

        addMouseMotionListener(this);

        setVisible(true);
    }

    @Override
    public void mouseMoved(MouseEvent e) {
        int x = e.getX();
        int y = e.getY();
        label.setText("X: " + x + " , Y: " + y);
    }

    @Override
    public void mouseDragged(MouseEvent e) {
        // Optional: handle drag if needed
    }

    public static void main(String[] args) {
        new MouseCoordinates();
    }
```

}

OUTPUT :



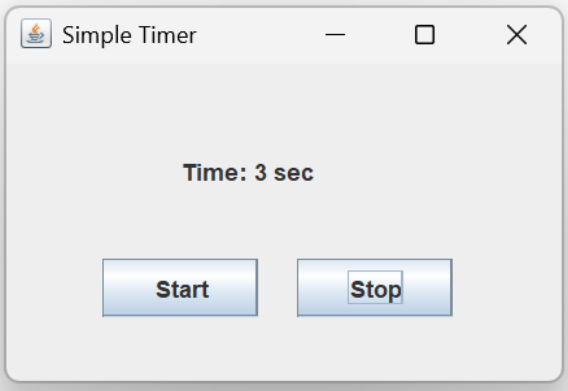
2. GUI Program to implement a simple Timer (using background events). Include a Start and Stop button to control the timer.

```
import javax.swing.*;
import java.awt.event.*;

public class SimpleTimer extends JFrame implements ActionListener {

    JLabel timeLabel;
    JButton startBtn, stopBtn;

    int seconds = 0;
    Timer timer;

    public SimpleTimer() {
        setTitle("Simple Timer");
        setSize(300, 200);
        setLayout(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        timeLabel = new JLabel("Time: 0 sec");
        timeLabel.setBounds(90, 40, 150, 30);
```

```
        add(timeLabel);

        startBtn = new JButton("Start");
        startBtn.setBounds(50, 100, 80, 30);
        add(startBtn);

        stopBtn = new JButton("Stop");
        stopBtn.setBounds(150, 100, 80, 30);
        add(stopBtn);

        // Timer fires every 1000 ms (1 second)
        timer = new Timer(1000, new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                seconds++;
                timeLabel.setText("Time: " + seconds + " sec");
            }
        });

        startBtn.addActionListener(this);
        stopBtn.addActionListener(this);

        setVisible(true);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == startBtn) {
            timer.start();
        } else if (e.getSource() == stopBtn) {
            timer.stop();
        }
    }

    public static void main(String[] args) {
        new SimpleTimer();
    }
}
```

OUTPUT :

```
c:\Users\PRASANNA\Desktop\AOOC-Practicals-4th sem\exp10>cd "c:\Users\PRASANNA\De
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
]
```

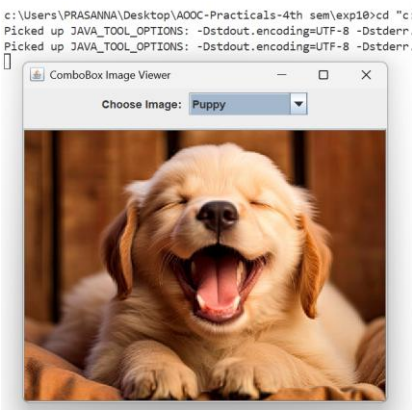


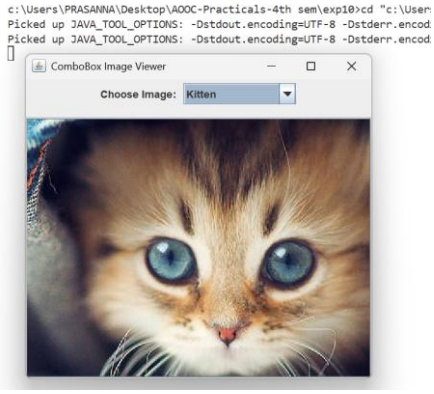
2. **Create a GUI with a JComboBox containing image names. On selection, display the corresponding image using a JLabel and ItemListener.**

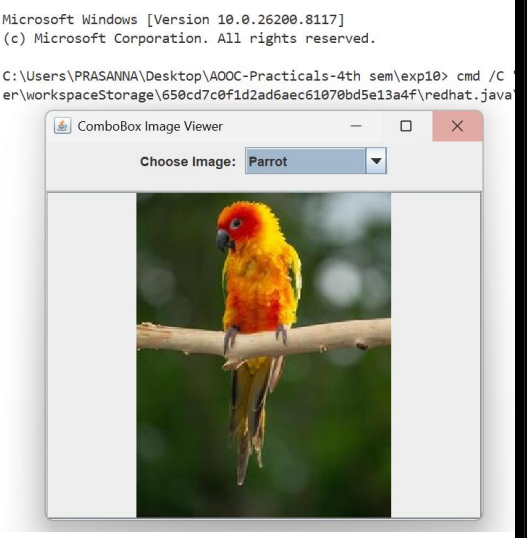
```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ItemEvent;
import java.awt.event.ItemListener;
import java.util.HashMap;
import java.util.Map;

public class ImageViewerApp extends JFrame {

    private JComboBox<String> imageComboBox;
    private JLabel imageLabel;
    private Map<String, String> imagePaths;

    public ImageViewerApp() {
        // Set up the main frame
        setTitle("ComboBox Image Viewer");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout(10, 10));

        // 1. Initialize our image data mapping (Name -> File Path)
        imagePaths = new HashMap<>();
        imagePaths.put("Select an image...", "");
```

```
        imagePaths.put("Puppy", "puppy.jpg");
        imagePaths.put("Kitten", "kitten.jpg");
        imagePaths.put("Parrot", "parrot.jpg");

        // 2. Create the JComboBox with the image names
        String[] imageNames = { "Select an image...", "Puppy", "Kitten", "Parrot" };
        imageComboBox = new JComboBox<>(imageNames);

        // 3. Create the JLabel to hold the image
        imageLabel = new JLabel("Please select an image above.", SwingConstants.CENTER);
        imageLabel.setPreferredSize(new Dimension(400, 300));

        // Optional: add a border for better visual presentation
        imageLabel.setBorder(BorderFactory.createLineBorder(Color.GRAY));

        // 4. Add the ItemListener to the JComboBox
        imageComboBox.addItemListener(new ItemListener() {
            @Override
            public void itemStateChanged(ItemEvent e) {
                // Ensure we only react to the new item being selected,
                // ignoring the deselection of the previous item.
                if (e.getStateChange() == ItemEvent.SELECTED) {
                    String selectedName = (String) e.getItem();
                    updateImage(selectedName);
                }
            }
        });

        // 5. Add components to the frame
        // Pad the layout a bit for aesthetics
        JPanel topPanel = new JPanel();
        topPanel.add(new JLabel("Choose Image: "));
        topPanel.add(imageComboBox);

        add(topPanel, BorderLayout.NORTH);
        add(new JScrollPane(imageLabel), BorderLayout.CENTER);

        // Pack and center the window on screen
        pack();
        setLocationRelativeTo(null);
    }

    /**
     * Helper method to update the JLabel's icon based on the dropdown selection.
     */
    private void updateImage(String imageName) {
```

```
        String filePath = imagePaths.get(imageName);

        if (filePath != null && !filePath.isEmpty()) {
            ImageIcon icon = new ImageIcon(filePath);

            // Check if the image was successfully loaded
            if (icon.getIconWidth() == -1) {
                imageLabel.setIcon(null);
                imageLabel.setText("Image not found: " + filePath);
            } else {
                imageLabel.setIcon(icon);
                imageLabel.setText(""); // Clear the text when an image is shown
            }
        } else {
            // Revert to default text if "Select an image..." is chosen
            imageLabel.setIcon(null);
            imageLabel.setText("Please select an image above.");
        }
    }

    public static void main(String[] args) {
        // Run the GUI creation on the Event Dispatch Thread (EDT) for thread safety
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                new ImageViewerApp().setVisible(true);
            }
        });
    }
}
```





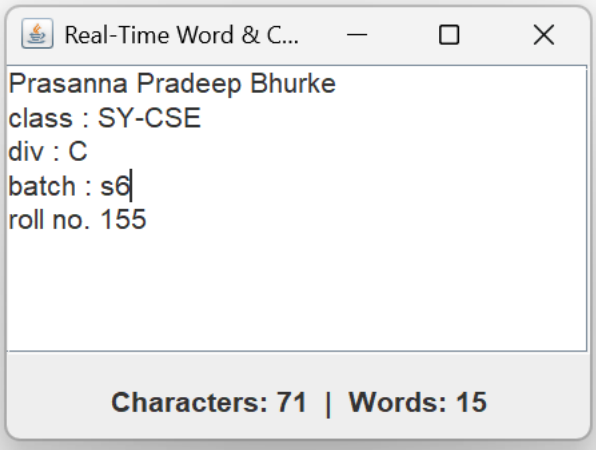


4. GUI with a JTextArea and a label. As the user types, show the character count and word count in real-time using a KeyListener

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.KeyEvent;
import java.awt.event.KeyListener;

public class WordCharacterCounter extends JFrame implements KeyListener {

    private JTextArea textArea;
    private JLabel countLabel;

    public WordCharacterCounter() {
        setTitle("Real-Time Word & Character Counter");
        setSize(500, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout(10, 10));

        // Initialize JTextArea
        textArea = new JTextArea();
        textArea.setLineWrap(true);
        textArea.setWrapStyleWord(true);
        textArea.setFont(new Font("Arial", Font.PLAIN, 14));

        // Add the KeyListener to the text area
        textArea.addKeyListener(this);

        // Initialize JLabel for counts
        countLabel = new JLabel("Characters: 0  |  Words: 0");
        countLabel.setFont(new Font("Arial", Font.BOLD, 14));
        countLabel.setBorder(BorderFactory.createEmptyBorder(5, 10, 10, 10));
        countLabel.setHorizontalAlignment(SwingConstants.CENTER);

        // Add components to the frame
        // A JScrollPane is highly recommended for text areas to allow scrolling
        add(new JScrollPane(textArea), BorderLayout.CENTER);
        add(countLabel, BorderLayout.SOUTH);

        setLocationRelativeTo(null); // Center the window on the screen
    }

    @Override
```

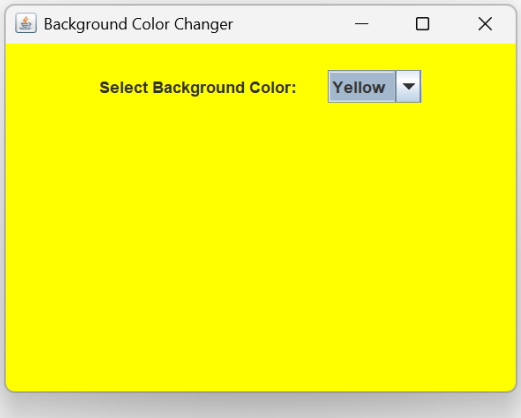
```
    public void keyTyped(KeyEvent e) {
        // Not used, but required by KeyListener interface
    }

    @Override
    public void keyPressed(KeyEvent e) {
        // Not used, but required by KeyListener interface
    }

    @Override
    public void keyReleased(KeyEvent e) {
        // keyReleased is ideal because the text component has already updated its
        // content
        String text = textArea.getText();

        int charCount = text.length();
        // Trim the text and split by whitespace to get the word count
        int wordCount = text.trim().isEmpty() ? 0 : text.trim().split("\\s+").length;

        countLabel.setText("Characters: " + charCount + "  |  Words: " + wordCount);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new WordCharacterCounter().setVisible(true));
    }
}
```

```
c:\Users\PRASANNA\Desktop\AOOC-Practicals-4th sem\ex
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8
```



5. Write Java GUI Program using Swing to change background on selecting color.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class BackgroundColorChanger extends JFrame {

    private JComboBox<String> colorDropdown;
    private JPanel mainPanel;

    public BackgroundColorChanger() {
        setTitle("Background Color Changer");
        setSize(400, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Create main panel
        mainPanel = new JPanel();
        mainPanel.setLayout(new FlowLayout(FlowLayout.CENTER, 20, 20));

        // Array of color names for the dropdown
        String[] colorNames = { "White", "Red", "Green", "Blue", "Yellow", "Orange", "Pink",
"Black" };
        colorDropdown = new JComboBox<>(colorNames);

        // Add an ActionListener to detect when a new color is selected
        colorDropdown.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                String selectedColor = (String) colorDropdown.getSelectedItem();
                changeBackgroundColor(selectedColor);
            }
        });

        // Add components to the panel
        mainPanel.add(new JLabel("Select Background Color: "));
        mainPanel.add(colorDropdown);

        // Add the main panel to the frame
        add(mainPanel);

        // Center the window on the screen
        setLocationRelativeTo(null);
    }
```

```
    // Helper method to convert String selection to an actual java.awt.Color
    private void changeBackgroundColor(String colorName) {
        switch (colorName) {
            case "Red":
                mainPanel.setBackground(Color.RED);
                break;
            case "Green":
                mainPanel.setBackground(Color.GREEN);
                break;
            case "Blue":
                mainPanel.setBackground(Color.BLUE);
                break;
            case "Yellow":
                mainPanel.setBackground(Color.YELLOW);
                break;
            case "Orange":
                mainPanel.setBackground(Color.ORANGE);
                break;
            case "Pink":
                mainPanel.setBackground(Color.PINK);
                break;
            case "Black":
                mainPanel.setBackground(Color.BLACK);
                break;
            default:
                mainPanel.setBackground(Color.WHITE);
                break;
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new BackgroundColorChanger().setVisible(true));
    }
}
```

```
c:\Users\PRASANNA\Desktop\AOOC-Practicals-4th sem\exp10>cd "c:\User
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoc
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoc
```